PROJECT REPORT • 01 / 30

Co-WingMan EDGE AI

A privacy-conscious conversational recovery assistant for NVIDIA Jetson Orin Nano

Document profile

Technical project report | Repository: Co-Wingman-AI/Co-Wingman-Edge-AI | Branch reviewed: main | Review date: 21 October 2026

Purpose

This document describes the product concept, architecture, implementation, operational controls, evidence of verification, limitations, and a practical validation roadmap. Its primary basis is the repository README, module structure, AGENTS.md, WORK_LOG.md, and linked operational documentation.

Scope and accuracy

The report distinguishes source-code behavior, documented operator observations, offline tests, and features that still require live Jetson, API, or AWS verification. It is not a claim of clinical efficacy, speaker identification, continuous full-transcript capture, or independent hardware certification.

Executive statement

Co-WingMan EDGE helps a person recover during a conversation by offering a short, usable phrase when explicitly requested or, in optional modes, after a detected pause. Audio processing occurs locally. A tightly bounded excerpt of recognized text can be sent to an approved UF NaviGator model, then strictly validated before display.

PREPARED FROM THE PROJECT SOURCE REPOSITORY

EXECUTIVE SUMMARY • 02 / 30

Project snapshot

Motivation, solution and engineering posture

Problem addressed

Conversation can become difficult when a speaker loses their place, needs clarification, or cannot promptly formulate a response. Conventional conversational assistants can demand full interaction, stream excessive data, or generate long replies. Co-WingMan instead narrows the interaction to a single, short recovery cue.

Proposed solution

A personal microphone captures short audio windows on a Jetson. Local NVIDIA NeMo ASR converts speech into text. A bounded context buffer selects recent fragments. The operator presses Enter or types :cue to request a phrase. Optional pause assistance can initiate a request under conservative timing constraints.

Engineering priorities

The implementation prioritizes minimal remote disclosure, small output, transparent operator control, conservative validation, and predictable resource use on an 8 GB embedded platform. Typed and fully synthetic offline modes support development when NVIDIA or network services are unavailable.

Current status

The repository contains a working Python application with multiple operating modes, extensive offline tests, and documented operator-reported Jetson microphone and NaviGator demonstrations. Sustained latency, recognition accuracy, automatic pause quality, optional TTS, and live AWS pathways remain areas for device-side measurement.

Reading guide

Thirty-page report navigation

Pages 1–10

1. Co-WingMan EDGE AI; 2. Project snapshot; 3. Reading guide; 4. Background and rationale; 5. Need, context, and boundaries; 6. Functional and nonfunctional specification; 7. Users and operating scenarios; 8. End-to-end component model; 9. NVIDIA Jetson Orin Nano 8 GB; 10. Information handling

Pages 11–20

11. Microphone pipeline; 12. NVIDIA NeMo inference; 13. Bounded and expiring memory; 14. Manual and optional automatic assistance; 15. UF NaviGator / NVIDIA Nemotron; 16. Schema, safety and presentation gates; 17. Selective local grounding; 18. Optional continuity across sessions; 19. Visible status and concise outputs; 20. Assets, trust boundaries, and controls

Pages 21–30

21. Predictable degraded behavior; 22. From checkout to edge runtime; 23. Verification layers; 24. Latency, memory, and throughput; 25. Disfluency shadow analysis; 26. Known constraints and tradeoffs; 27. Prioritized engineering follow-up; 28. How success should be demonstrated; 29. Phased next steps; 30. Project synthesis

Document conventions

“Implemented” means functionality represented in the reviewed code or documentation. “Offline verified” indicates recorded tests or simulations. “Operator reported” indicates a device result documented by a previous operator, not rerun as part of this report. “Proposed” indicates future work or a recommendation, not a current feature.

Repository authority

For active work, consult AGENTS.md and WORK_LOG.md first, followed by the README and task-specific documentation. The files in docs/ and RUN_ON_JETSON.md provide the detailed operational procedures.

INTRODUCTION • 04 / 30

Background and rationale

Assistive, narrowly scoped conversational support

Project context

Co-WingMan EDGE AI explores embedded, privacy-aware assistance during real conversations. The application is intended to avoid the cognitive overhead of a conventional chatbot. Rather than presenting a full conversation, it supplies one concise, actionable phrase that the user can say.

Distinctive interaction

The default experience places agency with the operator: capture recent language, request help with a keypress, read a very short cue, and return to the conversation. Audio is not uploaded. The cloud pathway is limited to text necessary for the cue.

Edge-plus-cloud partition

The system deliberately splits compute: microphone acquisition and automatic speech recognition happen on the Jetson; linguistic cue generation happens through UF NaviGator. This avoids keeping a large language model resident on an 8 GB Jetson, where prior local Nemotron experiments caused memory pressure and swap growth.

Design philosophy

Scope is intentionally constrained. The application does not promise diagnosis, emotion detection, covert listening, speaker recognition, or complete transcript archiving. These are product boundaries, not missing user-facing settings.

PROBLEM DEFINITION • 05 / 30

Need, context, and boundaries

What the system is intended to solve

User problem

A person may pause, lose wording, or need to ask for clarification while another person is speaking. Finding an appropriate response quickly can be more important than receiving an elaborate explanation. A cue must be short enough to read, remember, and say without significantly interrupting a live exchange.

Technical problem

Real-time conversational assistance must manage imperfect microphone capture, short or fragmented recognition results, uncertain speaker identity, network latency, and GPU/RAM limits. Sending a continuous transcript would expand privacy exposure without necessarily improving short-cue usefulness.

Functional boundary

The output is a recovery phrase with an intent classification: resume, clarify, or respond. The model may decline to offer a cue. It does not autonomously conduct a conversation or make binding claims about what a specific participant intended.

Non-goals

Not a medical device, anxiety classifier, biometric authentication system, reliable diarization engine, meeting recorder, or general-purpose autonomous agent. These distinctions shape input labeling, safety rules, persistence, and project evaluation.

GOALS AND REQUIREMENTS • 06 / 30

Functional and nonfunctional specification

Traceable engineering expectations

Functional requirements

FR-01: accept local microphone fragments or typed fallback. FR-02: bound and expire context. FR-03: request a cue explicitly. FR-04: permit optional pause triggering. FR-05: produce and validate one short structured phrase. FR-06: show terminal status and cue; optionally speak it locally. FR-07: provide synthetic offline demo and live health check.

Extended requirements

FR-08: opt-in 60-second rolling context and minute summaries. FR-09: optional S3 save/resume of compact summaries. FR-10: bounded portfolio retrieval from local JSON or S3. FR-11: numeric, content-free timing. FR-12: experimental shadow scoring isolated from production triggers.

Nonfunctional requirements

NFR-01: no raw audio to model endpoint. NFR-02: bounded request size and retention. NFR-03: strict allowlisted HTTPS model endpoint. NFR-04: validated compact output. NFR-05: resource-aware process lifecycle. NFR-06: no private text in application logs or work logs. NFR-07: usable typed mode without NVIDIA dependencies.

Acceptance strategy

Unit tests establish code invariants; integration mocks verify request and S3 boundaries; live hardware evaluation must separately establish memory safety, recognition usefulness, cue latency and quality, and long-duration stability.

Users and operating scenarios

Explicit control and consent

Primary operator

A consenting individual running the app on a personal microphone is the intended beneficiary. They need predictable cues and clear awareness when assistance is active. The display is intentionally visual by default to avoid unexpected audio output.

Secondary participants

Nearby participants may be audible to the microphone. Since no speaker identity is inferred, participant awareness and consent matter. Any text sent to the cloud can include language from either participant and must be constrained by the application.

Scenario A — manual recovery

The operator starts local listening, a brief exchange is recognized, and a pause or conversational difficulty occurs. The operator presses Enter. The system snapshots recent text, obtains or withholds a cue, and displays a short recovery phrase.

Scenario B — accessibility/fallback

The operator uses --typed when audio is unavailable, enters one or more short fragments, and invokes :cue. This mode supports deterministic demonstrations without sounddevice, NeMo, or GPU access.

Scenario C — rolling continuity

An operator opts into rolling mode to retain recent text across multiple manual cues. Optional minute summaries and S3 storage are separate choices rather than default persistence.

SYSTEM ARCHITECTURE • 08 / 30

End-to-end component model

Boundaries between edge and managed cloud

Edge input tier

A USB close-talk microphone feeds RAM-only audio acquisition. The speech worker produces text from bounded four-second windows. Audio acquisition and recognition are localized; the main application receives text fragments and events, not full files.

Application tier

Application orchestrates event handling, input commands, context expiration, cue requests, output lifecycle, rolling memory, and optional features. Context and portfolio retrieval apply explicit length limits before constructing an inference request.

Cloud inference tier

An HTTPS request targets UF NaviGator's approved OpenAI-compatible endpoint and the specified Nemotron model. Strict requested JSON schema is paired with mandatory local parsing, semantic heuristics, and length checks.

Output tier

Validated cues appear in a terminal display for a short interval. Optional TTS uses local NVIDIA models sequentially after ASR exits, reducing concurrent resource demand. Default cue mode pauses capture until :listen to avoid self-feedback.

Architecture principle

The primary trust boundary is between the Jetson and UF infrastructure. S3 introduces a second, opt-in cloud boundary for summary or portfolio artifacts, not raw speech.

Figure A. Detailed system architecture of Co-WingMan EDGE AI, showing local Jetson components, optional services, trust zones, and control boundaries.

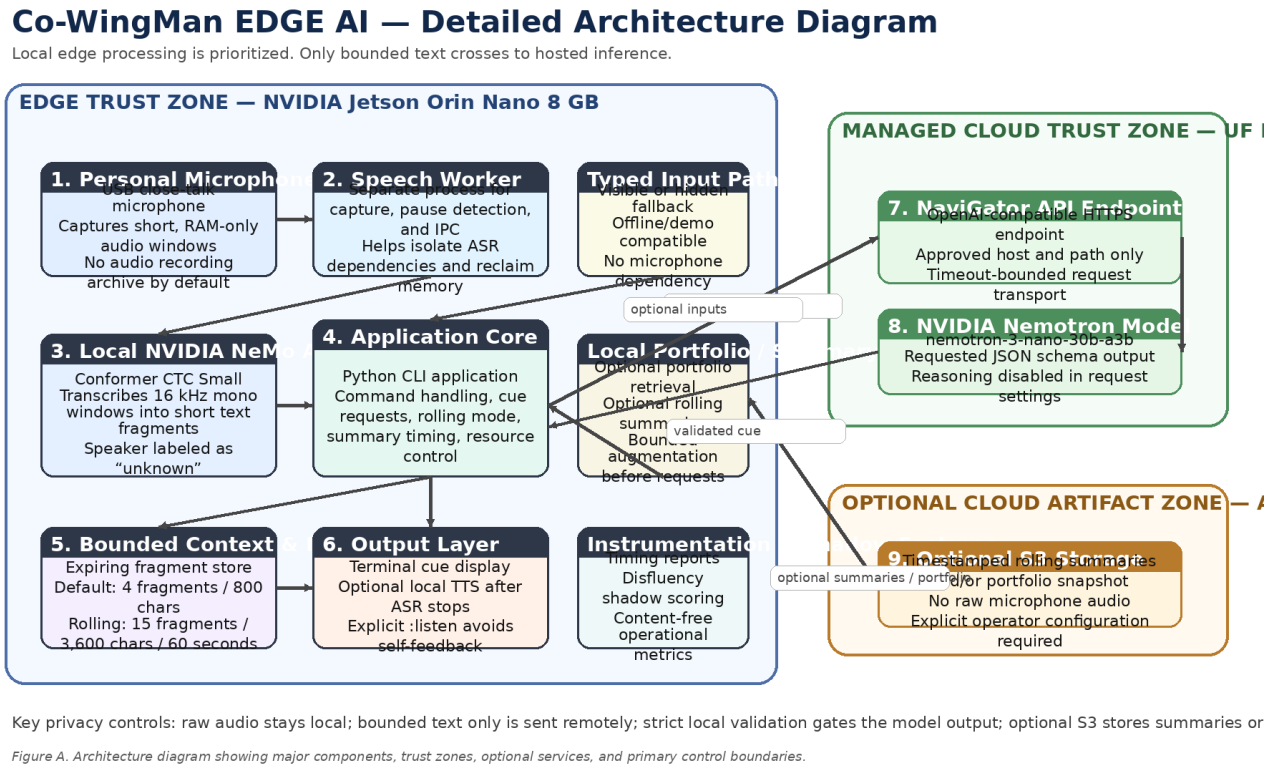


Figure A — Detailed Architecture Diagram

The following figure summarizes the major components, interactions, trust boundaries, and controls in this section.

HARDWARE ENVIRONMENT • 09 / 30

NVIDIA Jetson Orin Nano 8 GB

Embedded platform considerations

Target device

The project documents deployment on a Jetson Orin Nano 8 GB running an L4T 36.4.7 environment. A supported personal USB microphone provides live input; a terminal monitor provides default cue output.

Memory budget

The Jetson's shared memory is the major design constraint. The README records a prior local LLM attempt leaving roughly 294 MiB free and increasing swap. The production design therefore uses local ASR plus a remotely hosted LLM, not simultaneous local speech and local large-language-model inference.

Process separation

Speech and optional TTS operate in isolated processes. Startup applies a 2 GiB available-memory requirement; a 1 GiB runtime guard helps detect low-memory conditions, although neither guarantees freedom from OOM conditions.

Audio constraints

Capture uses 16 kHz mono windows, with local transcription. Microphone index listing and the COWINGMAN_AUDIO_DEVICE configuration support device selection. Operators should test placement, sensitivity, and acoustic conditions before live demonstrations.

Device validation

Physical performance requires checking thermal state, tegrastats/free memory, CUDA access, model restoration, audio I/O, and sustained sequential operation. Mac and x86 tests do not establish Jetson timing.

Information handling

From ephemeral samples to validated cue

Stage 1: capture

A local worker receives audio in bounded RAM windows. It measures acoustic activity and passes samples to the local ASR. Raw audio is not an API request field and is not intentionally persisted as an audio recording.

Stage 2: transcription

Recognized fragments are reduced to short text snippets. Microphone-origin text uses speaker label unknown. Capture time accompanies fragments so late ASR results do not refresh expired data as if they were new.

Stage 3: context selection

The context buffer enforces maximum fragment count, per-fragment characters, total characters, and time-to-live. At trigger time the app takes or snapshots recent context depending on mode.

Stage 4: augmentation

If explicitly enabled, a matching portfolio contributes up to two passages / 1,200 fact characters. Rolling requests may also include a bounded historical summary. These inputs remain separate from full audio and transcripts.

Stage 5: generation and disposal

Only selected text is sent via HTTPS. The response is parsed and validated, presented only if acceptable, and temporary request references are dropped. In default mode context is cleared; in rolling mode it expires naturally or by command.

Figure B. Data flow and lifecycle of audio/text, bounded context, optional augmentations, hosted inference, validation, presentation, and disposal.

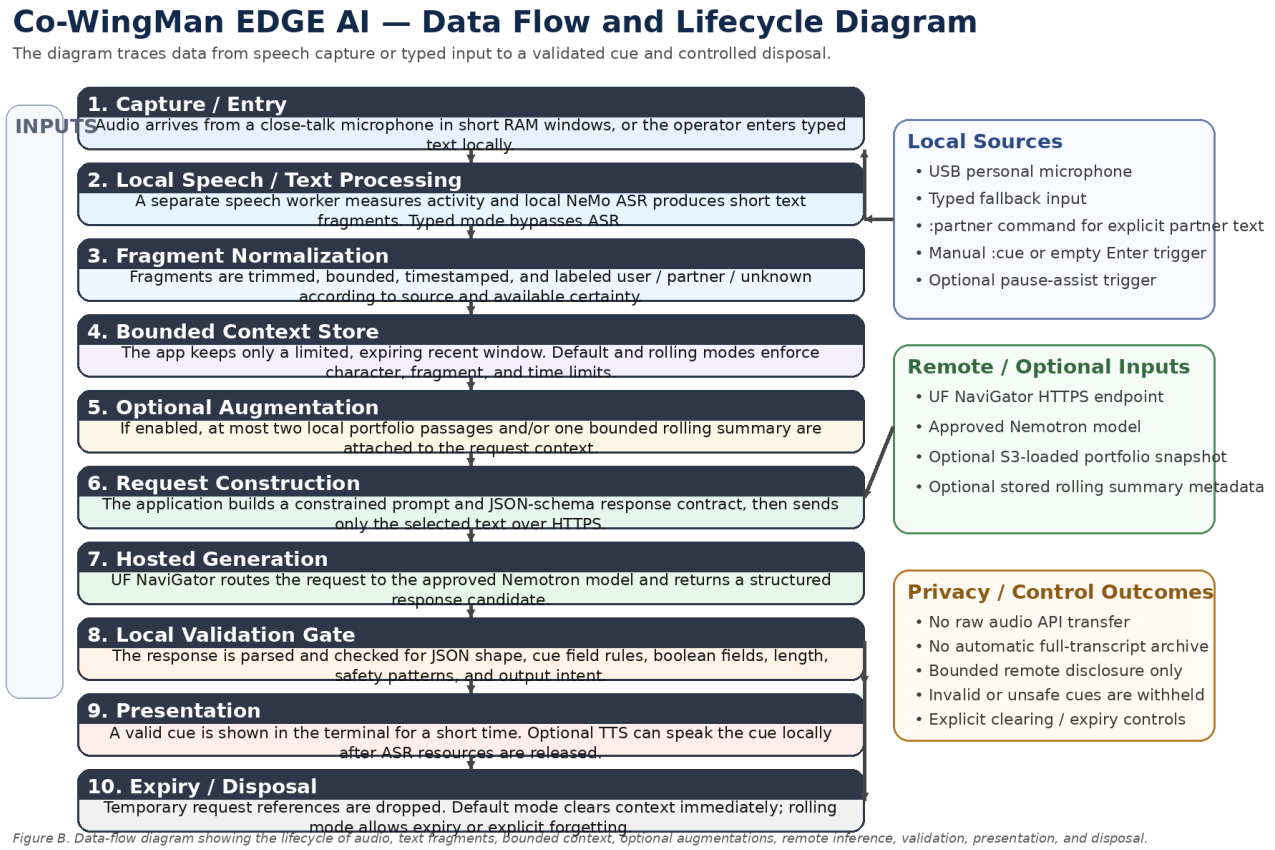


Figure B. Data-flow diagram showing the lifecycle of audio, text fragments, bounded context, optional augmentations, remote inference, validation, presentation, and disposal.

Figure B — Data Flow and Lifecycle Diagram

The following figure traces how audio or typed input becomes a bounded request, a validated cue, and then an expiring or discarded artifact.

AUDIO CAPTURE • 11 / 30

Microphone pipeline

Windowing and event-driven communication

Acquisition design

The prototype captures approximately four seconds of 16 kHz mono audio per recognition unit. Default capture pauses during transcription, making this a windowed pipeline rather than a fully streaming recognizer.

Rolling behavior

Rolling mode can capture while ASR and cue requests progress, with a bounded queue. If processing falls behind, older windows may be dropped and the operator receives an audio-gap notice. This is a deliberate bounded-memory tradeoff.

Timestamp integrity

The application carries original capture timestamps through worker communication and rejects stale fragments. This prevents delayed recognition from extending the apparent 60-second freshness window.

Energy-based pause support

The optional pause heuristic observes sound energy followed by sustained silence; it also requires a minimum speech period and applies a 30-second cooldown. Ordinary silence, background noise, and mic distance can generate false or missed triggers.

Implementation limitation

Window boundaries can cut words or split meaning across segments. Consecutive windows are merged when size limits permit in default mode, but no perfect conversational segmentation is claimed.

NVIDIA NeMo inference

Local speech-to-text without speaker claims

Model choice

The stated local model is NVIDIA Conformer CTC Small. The model runs on the edge rather than sending samples to UF. Setup scripts handle explicit downloading/preparation ahead of operation.

Worker isolation

ASR loads within a separate speech process and sends status/results to the app. This architecture helps reclaim memory when microphone processing is stopped and prevents mixing model internals into the main request path.

Recognition uncertainty

ASR output can omit words, mishear names, or merge voices. Short context mitigates exposure but also limits the model's understanding. The application does not calculate or promise real-world word error rate in its documentation.

Identity handling

No voiceprint enrollment, speaker verification, or diarization is implemented. Consequently, microphone-recognized text is unknown rather than user/partner. A separate :partner command allows explicitly labeled typed partner text for controlled demos.

Evaluation plan

Gather consented, local test scripts with varied mic distances and acoustic settings; measure word error rate, fragment completeness, event delay, and cue relevance. Do not retain private audio without an explicit new storage/privacy design.

Bounded and expiring memory

Privacy-preserving representation of recent speech

Default buffer

The nonrolling limit is four fragments, each at most 240 characters, with up to 800 total characters. Taking context for a manual request clears its store. The default interaction emphasizes near-term recovery, not long-term recall.

Rolling buffer

Opt-in rolling mode retains up to 15 fragments and 3,600 characters for a 60-second window. It supports repeated cues while listening remains active. Old records expire, and explicit clear/forget commands support operator intervention.

Design safeguards

Frozen data classes, strict value checks, bounded sequence lengths, and short time-to-live rules reduce accidental retention or oversized requests. Late captures are rejected based on original timestamps.

Context adequacy

A narrow excerpt helps reduce disclosure and inference cost, but it can omit the user's intent or a prior question. This tradeoff is fundamental. A cue must not invent facts or assign speech to a person when the context lacks evidence.

Operational distinction

Rolling memory is not automatically persistent memory. S3 summary storage is separately opt-in; default mode does not create a transcript archive.

Manual and optional automatic assistance

Operator-led interventions

Primary trigger

Enter on an empty line or :cue requests a cue. The implementation attempts to stop relevant speech capture in default mode, select bounded text, call the provider, validate output, and show the result.

Pause assistance

--pause-assist opts into a speech-energy/silence heuristic. It is explicitly described as possible pause detection, not emotional state inference. A 30-second cooldown limits repeat automatic triggers.

State transitions

Commands such as :listen, :pause, :clear, :assist on/off, and :quit let the operator control capture and context. Default mode pauses audio after a cue and requires explicit restart. Rolling mode differs by preserving recent context and continued listening.

Error behavior

No-context requests yield a benign prompt to supply fresh context. Audio errors fall back to typed entry where possible. Requests failing cloud or validation steps produce sanitized status rather than reproducing private model content.

Human factors

The explicit trigger is the recommended demonstration path because it avoids false pause detections and ensures the user intends to request a cue at that moment.

UF NaviGator / NVIDIA Nemotron

Strict remote request contract

Endpoint selection

The approved host/path is UF NaviGator's HTTPS OpenAI-compatible chat-completions endpoint. The repository specifies nemotron-3-nano-30b-a3b as the only approved remote model; silent substitution is not part of the runtime.

Credentials

A UF NaviGator Toolkit key is supplied by environment variable. The app does not load .env files and should never store tokens in committed documentation, traces, or command history.

Request shaping

Requests use structured JSON schema with cue, intent and should_offer; reasoning is disabled with a chat-template option and production output allocation is 256 tokens. Local code still validates response content rather than trusting server enforcement.

Transport defenses

The client constrains destination and path, disables redirects and environment proxies for its own HTTPS pathway, uses timeouts, and does not automatically retry failures or degrade to arbitrary JSON output.

Disclosure statement

UF receives the selected bounded text; local deletion does not imply deletion from UF logs or systems. Cloud retention and policy must be evaluated independently of the Jetson's in-process deletion strategy.

Schema, safety and presentation gates

Turning model text into a short speaking cue

Structured contract

A response must be a JSON object with exactly three properties: cue (string), intent (resume/clarify/respond), and should_offer (boolean). Duplicate keys, unexpected fields, invalid types, controls, and malformed JSON are rejected.

Length and form

Accepted cue text is restricted to at most 160 characters and 12 words, is nonempty, and avoids unsupported control characters. The requested API schema encodes similar constraints, but local checking remains the final gate.

Safety heuristic

The validator rejects a set of disallowed patterns including URLs, markup-like characters, credential language, certain harmful instructions and medical/psychological terms. This is a defensive heuristic, not a complete moderation or semantic safety classifier.

Oversized responses

An otherwise-valid offered cue that only violates length can undergo one bounded shortening call to the same model, sharing the original request deadline. The final result must independently pass constraints; invalid or declined shortening is withheld.

Fallback policy

A manual original-model decline can sometimes use a locally grounded portfolio phrase, with clear UI labeling. Automatic declines and shortening declines remain respected; the system must not convert refusals into automatic content.

Selective local grounding

Relevant facts without full personal-history upload

Purpose

Portfolio facts can help the cue respond to a question about projects, education, or experience instead of inventing an answer. The source is an explicitly imported local JSON file or a separately requested S3 startup download.

Retrieval boundary

The app selects at most two relevant passages and sends up to 1,200 fact characters alongside the much smaller conversational excerpt. Retrieval is local after the profile is loaded; it is not an open internet search during a conversation.

No-match behavior

With portfolio mode active, zero matching passages stops the cue request before an API call. The :check command can show local match counts so an operator can decide whether to supply a more specific short question.

Lifecycle and risk

Unlike ephemeral conversation text, the profile persists locally until explicitly removed. Facts are potentially sensitive, so operators should review them before import and use least-privilege filesystem/cloud access.

Documentation

See docs/PORTFOLIO.md and docs/AWS_SUMMARIES.md for format, import, upload and scope. Portfolio retrieval does not compensate for absent speaker identification.

Optional continuity across sessions

Summary persistence without full transcript archival

Local rolling summary

A separate approximately minute-based timer can ask NaviGator for a bounded summary of recent text. Without cloud storage, this summary remains in RAM and does not create a durable conversation archive.

S3 option

--summary-s3 enables rolling mode and lets the application save timestamped summary JSON to an operator-selected private S3 prefix. --resume-summary can retrieve the latest record or an identified archive window.

Privacy distinction

Stored records are compact, model-generated summaries—up to about 1,200 characters each—not raw audio or a complete transcript. Nevertheless, a summary can still reveal personal or sensitive information and must be handled as potentially private data.

AWS responsibilities

Bucket policy, credentials, region, encryption, access logging and lifecycle expiration belong to the operator's AWS configuration. The repository provides a setup helper and guides but does not auto-provision or silently upload by default.

Limitations

Summary generation is lossy, meaning context can be omitted or distorted. The work log records mocked S3 tests rather than verified live AWS upload/reload in the cited development environments.

Visible status and concise outputs

Accessible, controlled operator feedback

Terminal experience

The terminal shows component status, diagnostic categories, and one brief cue. Input visibility is configurable; typed mode shows entry by default, while --hide-input can conceal it.

Cue lifecycle

The on-screen cue is short-lived, clearing after approximately 20 seconds. Redirected output does not include cue text, intentionally reducing accidental capture in pipes and files.

Audio output

Optional --tts uses sequential local NVIDIA FastPitch and HiFi-GAN. ASR exits before TTS begins and FastPitch is unloaded before HiFi-GAN, reducing simultaneous model residency on the Jetson.

Feedback safety

In default mode audio capture stays paused after a cue until :listen, helping prevent the device from recognizing its own spoken output as conversational input. Rolling mode instead favors continuity and excludes TTS.

Design tradeoffs

Visual output is the safe default, but a terminal is less discreet than a wearable display. Future output-device changes should preserve the same cue lifetime, visibility, and no-unintended-logging rules.

Assets, trust boundaries, and controls

Protection by minimization rather than secrecy alone

Protected assets

Microphone samples, text fragments, selected conversation context, portfolio facts, summary records, API credentials, and cue output have different sensitivity and retention paths.

Local boundary

Audio remains in the speech process and is not written as a recording by normal operation. In-memory buffers expire or are cleared. This is not forensic zeroization; operating-system swap, crash dumps, terminal screenshots and external capture may still retain traces.

Remote boundary

UF receives bounded text through a restricted HTTPS endpoint. An optional private AWS bucket separately receives summaries or stores a portfolio snapshot. Each cloud service has independent retention and access policies.

Controls and gaps

Use a consenting personal microphone; never claim biometric enrollment. Avoid private prompt/cue/transcript logging. Protect environment credentials. Apply least privilege to S3. Test redirection, error diagnostics and output sinks for unintended disclosure.

Threat scenarios

Potential threats include a misplaced mic, unintended bystander capture, transcript leakage in logs, overly broad S3 IAM, API key exposure, endpoint redirection, and prompt injection within spoken text. The short-context and validation layers reduce—but cannot eliminate—such risks.

Predictable degraded behavior

No unsafe silent fallback

Audio and model failures

If audio devices disappear or NeMo fails to load, the app reports a sanitized error and typed interaction remains an option. A low-memory guard can skip speech operations or TTS rather than risking uncontrolled concurrent allocation.

Service failures

Authentication errors, model denial, timeout, rate limit and bad response structure produce diagnostic categories without dumping prompts or model replies. Normal request transport errors are not automatically retried.

Validation failures

Malformed, unsafe, or declined cue candidates are withheld. Only specific otherwise-valid length failures qualify for a single repair request; a second invalid result cannot trigger recursive generation.

Rolling queue behavior

If a window queue is saturated, dropping older windows is explicitly reported. Capture timestamps guard TTL interpretation. This favors bounded operation over a misleading claim of gap-free audio.

Recovery procedure

Use typed/offline mode to isolate network/hardware; run --health-check for approved model access; use --list-audio for microphone indexes; review staged Jetson dependency/model checks for speech restoration.

From checkout to edge runtime

Reproducible setup paths

Developer smoke test

Clone the repository, then run python3 -m cowingman --offline --demo. For interactive no-hardware behavior, use python3 -m cowingman --typed --offline. Both enable early testing without an API key or NVIDIA runtime.

Live hosted model setup

Obtain a UF NaviGator Toolkit key with access to the exact approved model. Set NAVIGATOR_TOOLKIT_API_KEY in the current environment, execute --health-check with synthetic text, then try python3 -m cowingman --typed before introducing microphone and GPU complexity.

Jetson setup order

Follow RUN_ON_JETSON.md for its pinned or compatible NVIDIA environment. Restore speech preprocessing dependencies first, validate ASR model restore on CPU, confirm CUDA inference, list microphone devices, then run the microphone-to-cue sequence.

Optional services

Install the cloud extra for boto3-based S3 features and separately configure AWS region, profile, encryption and bucket permissions. Do not turn on rolling/S3 persistence solely as a deployment convenience.

Rollback

Preserve working state, record source commit, and use tested commits/fast-forward reconciliation rather than destructive clean/reset or force-pushing shared history.

Verification layers

Evidence, not assumptions

Offline unit coverage

The latest WORK_LOG.md entry records 128 tests passing across the expanded project. Tests exercise context limits, model response validation, request boundaries, timed expiry, queue management, command behavior, summary rules, mocked cloud persistence, and privacy-sensitive error handling.

Continuous integration

GitHub Actions is documented running offline tests across Python 3.10, 3.12 and 3.14 at different milestones. Ruff lint/format checks and repository hygiene scanning supplement the test harness.

Synthetic integration

The offline demo verifies an end-to-end mock pathway without audio hardware or the remote model. The real --health-check uses synthetic text to exercise UF access and structured response assumptions without transmitting a live conversation.

Evidence caveat

A test passing on a macOS or Linux development machine does not establish Jetson CUDA performance, actual microphone reliability, hosted model compliance, AWS credentials, or conversational usefulness.

Suggested release gates

Require green CI, successful offline demo, clean privacy/hygiene checks, reviewed changed output schemas, a timed physical Jetson smoke run, and explicit user-consent checks for new data persistence.

Latency, memory, and throughput

Measure each expensive stage independently

Current instrumentation

The optional --timings file records content-free numeric stage measurements for capture, ASR load/inference, worker startup, HTTPS wait/read, validation, total cue time, summary, optional TTS, and S3.

Primary performance factors

Four-second capture windows establish a baseline interaction delay. ASR startup/reload, CUDA scheduling, network round trip, and model generation compound time-to-cue. Optional length repair adds another remote request within the existing timeout.

Memory management

Separating workers and avoiding resident local Nemotron are central resource strategies. GPU memory and system RAM need to be observed under repeated sessions, especially when NeMo or TTS models are repeatedly restored.

Token-budget research

A synthetic benchmark compares four cue token budgets. Production remains at 256 output tokens until live comparison validates that a lower budget preserves correct, well-formed cues and does not increase refusal/truncation.

Recommended measurements

Collect median and tail time-to-cue, time to first accepted fragment, peak RAM/GPU use, queue drops per hour, failure rate, network response categories, and subjective cue usefulness. Never include transcript or cue text in timing reports.

EXPERIMENTAL RESEARCH TRACK • 25 / 30

Disfluency shadow analysis

Research isolated from production intervention

Motivation

Speech disfluency scoring is explored as an experimental local signal. The goal is to characterize possible timing or hesitation-related patterns, not to diagnose anxiety, emotions, clinical status, or identity.

Isolation

The --disfluency-shadow option requires rolling microphone mode and timing instrumentation. It computes local shadow scores but does not initiate cue requests or change the production pause heuristic.

Implementation notes

The documented adapter uses pinned model weights and a Keras/PyTorch CPU path; warmup occurs before capture. Fixed review markers support examining performance without storing recordings.

Evidence status

The work log records a temporary CPU test that loaded actual weights and scored synthetic windows, including in a network-disabled speech environment. This demonstrates adapter compatibility, not accuracy on conversational audio or Jetson latency.

Research protocol

Before promotion, define a consenting evaluation population, annotation protocol, false-trigger costs, subgroup analysis, data-retention plan, and a separate safety review. No inference of medical or mental-health state should be introduced by implication.

Known constraints and tradeoffs

Conservative interpretation of current capabilities

Speech quality

Four-second windowed ASR may miss parts of words or multi-speaker exchanges. Without diarization, fragments can be attributed to nobody specifically, constraining cue correctness.

Time-to-cue

Four-second acquisition, local inference, remote round trip, validation, and optional second model call may delay a cue enough to reduce real-time value. Published source notes are not a substitute for measured P95 latency on the target Jetson.

Privacy exposure

Only text is uploaded, but that text may still carry identifiable content or another speaker's words. An S3 summary or portfolio record persists beyond the minute window. Device RAM release is not secure erasure.

Inference reliability

Strict schemas and pattern checks cannot guarantee semantic correctness. The model may misunderstand context, echo a question, decline a useful answer, or generate a valid but unhelpful phrase.

Operational complexity

NeMo, CUDA, Librosa/setuptools compatibility, microphone selection, API permissions, AWS IAM, and Jetson memory can introduce environmental failures. The staged validation sequence is therefore essential.

Prioritized engineering follow-up

Risk ownership should be assigned before field use

High: bystander privacy

Risk: private words from nearby people appear in selected context. Mitigation: informed participation, close-talk personal mic, explicit trigger default, bounded snippets, no full transcript, and clear notices.

High: unusable cue timing

Risk: generated phrase arrives too late to help. Mitigation: profile every stage on device, optimize model warmup and prompts only after measurements, show request progress, preserve quick typed/manual fallback.

High: memory exhaustion

Risk: overlapping CUDA/ASR/TTS operations destabilize the Jetson. Mitigation: sequential model use, isolated processes, RAM guards, tegrastats monitoring, soak testing, safe default visual cues.

Medium: cloud/output failures

Risk: unavailable UF endpoint or strict-schema rejection blocks assistance. Mitigation: synthetic health-check, sanitized status, no arbitrary model fallback, clear operator recovery guidance.

Medium: summary mistakes

Risk: model-generated rolling summary distorts context. Mitigation: short TTL, clear labeling, manual review in pilots, no attribution claims, no automatic durable summary storage.

Medium: maintenance drift

Risk: documentation and tested configuration diverge. Mitigation: update WORK_LOG.md, CI across Python versions, preflight checks, and committed reproduction procedures.

EVALUATION FRAMEWORK • 28 / 30

How success should be demonstrated

Functional, performance, privacy and human evaluation

Functional outcomes

For each scripted use case, verify the expected cue action: offered or withheld, under all length bounds, matching a sensible intent, and free of unsupported claims. Record only scored outcomes and redacted examples.

Performance outcomes

Measure acceptance of fragments, request completion/failure, median/P95 total latency, model and ASR contributions, memory high-water mark, CPU/GPU temperatures, and dropped capture windows.

Privacy outcomes

Inspect HTTPS requests to confirm no audio upload; verify default no transcript persistence; validate no cue in redirected stdout or logs; audit optional S3 objects, encryption, IAM and deletion lifecycle.

Human-centered evaluation

Run an informed-consent pilot with scripted conversations, collecting ratings of timing, clarity, naturalness, relevance, cognitive burden, and unwanted assistance. Compare manual assistance against pause-assist separately.

Stop conditions

Stop a trial after accidental private disclosure, repeated misleading cues, significant OOM instability, high false automatic-trigger rates, or unclear participant consent. Evaluate opt-in features independently of baseline safety.

IMPLEMENTATION ROADMAP • 29 / 30

Phased next steps

Evidence-driven development priorities

Phase 1 — establish baseline

On physical Jetson, rerun the staged speech environment checks in RUN_ON_JETSON.md. Verify microphone capture, CUDA NeMo, typed live request, default cue, and clean process shutdown. Capture content-free timings.

Phase 2 — harden conversation loop

Measure ASR coverage and cue delay under quiet/noisy tests. Adjust audio thresholds and window behavior only with controlled benchmarks. Test interruption and microphone disconnect recovery.

Phase 3 — validate opt-in services

Separately exercise rolling capture, 60-second expiration, minute summaries, S3 encrypted save/resume, and portfolio S3 load under real AWS credentials. Confirm permissions and disclosure notices.

Phase 4 — measure usability

Run consented scripted sessions evaluating whether a 12-word recovery cue is timely and useful. Gather false-assist and unnecessary-assist feedback without storing full real conversations.

Phase 5 — polish presentation

Improve installation reproducibility, evidence tables, a visual status interface or discreet display if desired. Preserve the strict cue lifecycle and the explicit manual-trigger default throughout.

Project synthesis

Repository-grounded conclusion and source map

Conclusion

Co-WingMan EDGE demonstrates a focused embedded-assistance architecture: local speech capture and ASR, minimal text transfer to a controlled hosted Nemotron model, short validated cues, and intentionally limited retention. Its strongest design characteristic is not model sophistication alone but the explicit boundaries on who controls triggering, what is sent, and how output is accepted.

What has been demonstrated

The repository reports broad offline coverage, synthetic demo execution, lint and hygiene checks, and selected operator-reported live Jetson/NaviGator runs. It also contains implementation paths for rolling memory, optional S3, portfolio grounding, profiling and local shadow scoring.

What remains to establish

A defensible effectiveness claim requires independently reproducible Jetson latency and memory measurements, ASR and cue-quality evaluation, live optional-cloud validation, and consent-centered user tests. No clinical or speaker-recognition capability should be inferred.

Primary project sources

Repository: https://github.com/Co-Wingman-AI/Co-Wingman-Edge-AI ; README.md ; AGENTS.md ; WORK_LOG.md ; RUN_ON_JETSON.md ; docs/HANDOFF.md ; docs/PRIVACY.md ; docs/SPEECH.md ; docs/PORTFOLIO.md ; docs/AWS_SUMMARIES.md ; docs/PROFILING_AND_SHADOW.md ; docs/PRESENTATION.md .

Glossary

ASR = automatic speech recognition; CTC = connectionist temporal classification; LLM = large language model; TTL = time to live; TTS = text-to-speech; S3 = Amazon Simple Storage Service; cue = a concise, validated phrase intended for the operator to speak.

Risk Identification, Assessment, and Mitigation

This qualitative risk register evaluates threats to the current Co-WingMan EDGE AI prototype. Likelihood and impact ratings are engineering estimates rather than measured probabilities.

ID	Risk	Likelihood	Impact	Level	Mitigation
R01	Speech recognition errors	High	High	High	Evaluate WER, close-talk microphone, noise handling.
R02	Response latency	High	High	High	Measure end-to-end latency, ASR and API stages; optimize bounded inference.
R03	Jetson memory limitations	Medium	High	High	Isolate speech workers, manage sequential model loading, monitor RAM.
R04	Internet or API failure	Medium	High	High	Use clear errors, health checks and typed/offline demonstration fallback.
R05	Incorrect conversational cues	High	High	High	Validate schema and output; test contextual relevance with users.
R06	Speaker identification limitations	High	Medium	High	Retain unknown labels; evaluate consent-based diarization.
R07	False pause detection	High	Medium	High	Keep opt-in, tune thresholds and cooldowns, retain manual trigger.
R08	Sensitive text exposure	Medium	High	High	Bound transmitted context, avoid content logging, require informed consent.
R09	Jetson dependency incompatibilities	Medium	High	High	Pin dependencies and test CUDA, NeMo and audio on device.
R10	Context loss	High	Medium	High	Evaluate rolling windows and summary fidelity.
R11	S3 exposure and retention	Medium	High	High	Least-privilege IAM, encryption, bucket restrictions and lifecycle rules.
R12	Unreliable disfluency scoring	High	Medium	High	Keep shadow-only; test representative users and environments.
R13	Microphone disconnect or battery loss	Medium	Medium	Medium	Surface device status and preserve typed fallback.
R14	Insufficient real-	High	High	High	Repeated physical Jetson trials,

	world validation				sustained-load and user evaluation.
R15	Third-party service dependency	Medium	High	High	Health checks, compatibility testing, approved endpoint contract.

Risk Level Interpretation

High: prioritize mitigation and verification before broader deployment. Medium: monitor and address during iterative testing. Low: document and periodically review.

Overall Risk Assessment

Current deployment-readiness risk is HIGH, while the system remains appropriate for controlled prototype research. Priority verification areas are speech accuracy, end-to-end responsiveness, cue usefulness, privacy controls, and sustained Jetson reliability. The documented 128 offline tests and operator-reported live demonstrations do not establish production-grade performance.